

Linux · Docker · Git 명령어 레퍼런스

실제로 서버 운영하면서 쓴 명령어 전체 정리

madayblog.com · 2026.04.06

목차

1. Linux 기본 — 프로세스, 메모리, 디스크
2. Linux 네트워크 — 포트, SSH, curl
3. Linux 시스템 — systemctl, journalctl, cron
4. Linux 파일 — 권한, 복사, 압축
5. Docker 기본 — 컨테이너 조회, 실행
6. Docker 고급 — 로그, 내부 실행, 볼륨
7. Docker Compose
8. Git
9. 기타 도구 — curl, npm, weasyprint
10. 자주 쓰는 조합 패턴

1. Linux 기본 — 프로세스 · 메모리 · 디스크

ps — 실행 중인 프로세스 목록

명령어	설명
ps aux	전체 프로세스 목록 (a=모든 유저, u=사용자 형식, x=터미널 없는 프로세스 포함)
ps aux --sort=-%mem	메모리 사용량 많은 순으로 정렬
ps aux --sort=-%cpu	CPU 사용량 많은 순으로 정렬
ps aux grep nginx	nginx 관련 프로세스만 필터

```
USER      PID %CPU %MEM    VSZ   RSS  COMMAND
ubuntu   1234   0.1   0.5  12345   6789   node server.js
```

PID(프로세스 ID)는 프로세스를 종료할 때 사용. kill 명령어에 넘김.

kill / pkill — 프로세스 종료

명령어	설명
-----	----

<code>kill 1234</code>	PID 1234 프로세스에 종료 신호(SIGTERM) 전송
<code>kill -9 1234</code>	강제 종료 (SIGKILL, 즉시 죽임)
<code>pkill -f mongodb-mcp-server</code>	명령어 이름으로 일치하는 모든 프로세스 종료

```
pkill -f mongodb-mcp-server # 이름에 "mongodb-mcp-server" 포함된 프로세스 전부 종료
```

free — 메모리 사용량

명령어	설명
<code>free -h</code>	메모리 사용량 (h=사람이 읽기 좋은 단위, GB/MB)

```

total      used      free      shared  buff/cache   available
Mem:      23Gi    3.8Gi    2.1Gi        5Mi        17Gi        19Gi
Swap:      0B          0B          0B

```

available = 실제로 쓸 수 있는 메모리 (free + buff/cache 일부). buff/cache는 OS가 성능을 위해 캐시로 쓰는 것이라 필요하면 돌려줌.

df — 디스크 사용량

명령어	설명
<code>df -h</code>	파일시스템별 디스크 사용량
<code>df -h /home</code>	특정 경로의 디스크만 확인

```

Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   12G   38G   24% /

```

2. Linux 네트워크 — 포트 · SSH · curl

ss — 열린 포트 / 소켓 확인

명령어	설명
ss -tuln	열린 포트 목록 (t=TCP, u=UDP, l=listening, n=숫자로 표시)
ss -tulnp	+p 옵션: 어떤 프로세스가 포트를 쓰는지도 표시

```
Netid  State  Local Address:Port
tcp    LISTEN  0.0.0.0:80      # 0.0.0.0 = 외부 접근 가능
tcp    LISTEN  127.0.0.1:27017 # 127.0.0.1 = 로컬호스트만 접근 가능
```

0.0.0.0은 모든 IP에서 접근 가능. 127.0.0.1은 서버 자신만 접근 가능 (외부 차단).

ssh — 원격 서버 접속 / 터널

명령어	설명
ssh ubuntu@서버IP	기본 접속
ssh -i 키파일 ubuntu@서버IP	특정 키 파일로 접속
ssh -L 5601:localhost:5601 ubuntu@IP -N	SSH 터널: 로컬 5601포트 → 서버 5601포트

```
# Kibana SSH 터널 (로컬 PC에서 실행)
ssh -L 5601:localhost:5601 -i C:/ocikey/privateKey/ssh-key.key ubuntu@129.154.55.20 -N
# -N = 명령 실행 없이 터널만 유지, 커서가 멈추는 게 정상
```

ssh-keygen — SSH 키 생성

```
ssh-keygen -t ed25519 -C "raspberrypi"
# 이후 Enter 3번 (기본 경로, 빈 패스프레이즈)
cat ~/.ssh/id_ed25519.pub # 공개키 출력 → 서버 authorized_keys에 추가
```

curl — HTTP 요청

명령어	설명
-----	----

<code>curl https://사이트</code>	GET 요청, 응답 본문 출력
<code>curl -s URL</code>	진행 표시줄 없이 조용히
<code>curl -s URL -o /dev/null -w "%{http_code}"</code>	상태코드만 출력
<code>curl -s ifconfig.me</code>	내 서버의 공인 IP 확인
<code>curl -X POST URL -H "Content-Type: application/json" -d '{}'</code>	POST 요청

블로그가 제대로 응답하는지 확인

```
curl -s https://madayblog.com -o /dev/null -w "%{http_code}"
```

200 이면 정상

3. Linux 시스템 — systemctl · journalctl · cron

systemctl — 서비스 관리

명령어	설명
<code>systemctl status docker</code>	docker 서비스 상태 확인
<code>systemctl start nginx</code>	서비스 시작
<code>systemctl stop nginx</code>	서비스 중지
<code>systemctl restart nginx</code>	서비스 재시작
<code>systemctl enable cron</code>	부팅 시 자동 시작 등록
<code>systemctl disable cron</code>	자동 시작 해제
<code>systemctl list-units --type=service --state=running</code>	실행 중인 서비스 전체 목록

journalctl — 시스템 로그

명령어	설명
<code>journalctl -u docker</code>	docker 서비스 로그 전체
<code>journalctl -u docker --since "1 hour ago"</code>	최근 1시간 로그만
<code>journalctl -u docker -f</code>	실시간 로그 (follow)
<code>journalctl --since "2026-04-06" --until "2026-04-07"</code>	날짜 범위로 조회

```
journalctl -u docker --since "1 hour ago" | grep -i error
# docker 최근 1시간 로그에서 error 포함된 줄만 필터
```

cron — 정해진 시간에 자동 실행

```
# cron 설치 (Ubuntu)
sudo apt-get install -y cron
sudo systemctl enable cron
sudo systemctl start cron
```

```
# crontab 편집
crontab -e

# cron 표현식: 분 시 일 월 요일
0 3 * * * /home/makecool0/backup/pi_backup.sh >> ~/backups/cron.log 2>&1
# 매일 새벽 3시에 백업 스크립트 실행, 로그를 파일에 저장

0 4 * * * pkill -f mongodb-mcp-server
# 매일 새벽 4시에 좀비 프로세스 정리
```

위치	값	예시
분	0-59	30 = 30분
시	0-23	3 = 새벽 3시
일	1-31	* = 매일
월	1-12	* = 매월
요일	0-7 (0,7=일)	1 = 월요일

4. Linux 파일 — 권한 · 복사 · 압축

파일/디렉토리 기본

명령어	설명
<code>ls -la</code>	파일 목록 (l=상세, a=숨김파일 포함)
<code>mkdir -p /path/to/dir</code>	디렉토리 생성 (-p: 중간 경로도 자동 생성)
<code>cp</code> 원본 대상	파일 복사
<code>mv</code> 원본 대상	파일 이동 / 이름 변경
<code>rm -rf</code> 경로	강제 삭제 (주의: 복구 불가)
<code>find /path -name "*.log"</code>	파일 검색

chown — 파일 소유자 변경

```
sudo chown -R 1000:1000 ./elasticsearch_data
# -R: 하위 디렉토리까지 재귀 적용
# 1000:1000 = UID:GID (Elasticsearch 컨테이너 내부 유저)
```

Docker 컨테이너가 마운트된 디렉토리에 못 쓴다면 대부분 소유자 불일치 문제. 컨테이너 내부 프로세스의 UID로 맞춰줘야 함.

tar — 압축/해제

명령어	설명
<code>tar -czf 파일.tar.gz 폴더/</code>	gzip 압축 (c=생성, z=gzip, f=파일명)
<code>tar -xzf 파일.tar.gz</code>	압축 해제 (x=추출)
<code>tar -tzf 파일.tar.gz</code>	내용 목록만 확인

```
tar -czf /tmp/mablog_backup.tar.gz /tmp/mablog_dump_host/
# /tmp/mablog_dump_host/ 폴더를 gzip 압축
```


scp — SSH로 파일 전송

로컬 → 원격

scp 로컬파일 ubuntu@서버IP:/원격경로/

원격 → 로컬

scp ubuntu@서버IP:/원격파일 ./로컬경로/

-i 옵션으로 키 파일 지정

scp -i ~/.ssh/id_ed25519 /tmp/backup.tar.gz makecool0@172.30.1.57:/home/makecool0/backups/

5. Docker 기본

컨테이너 조회

명령어	설명
<code>docker ps</code>	실행 중인 컨테이너 목록
<code>docker ps -a</code>	중지된 것 포함 전체 목록
<code>docker ps --format "table {{.Names}}\t{{.Status}}"</code>	이름과 상태만 보기 좋게 출력

리소스 모니터링

명령어	설명
<code>docker stats</code>	실시간 CPU/메 모리 사 용량 (Ctrl+C 로 종료)
<code>docker stats --no-stream</code>	현재 순 간 스냅 샷만 출 력
<code>docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.CPUPerc}}"</code>	이름/메 모리/ CPU만 출력

NAME	MEM USAGE / LIMIT	CPU %
mablog_nextjs	69.3MiB / 23.42GiB	0.10%
mablog_mongodb	77.73MiB / 23.42GiB	0.42%

이미지 관리

명령어	설명
<code>docker pull nginx:stable-alpine</code>	이미지 다운로드

<code>docker images</code>	로컬에 있는 이미지 목록
<code>docker rmi</code> 이미지명	이미지 삭제

컨테이너 시작/중지

명령어	설명
<code>docker start</code> 컨테이너명	중지된 컨테이너 시작
<code>docker stop</code> 컨테이너명	컨테이너 중지 (graceful)
<code>docker restart</code> 컨테이너명	재시작
<code>docker rm</code> 컨테이너명	컨테이너 삭제 (중지 후 가능)

6. Docker 고급 — 로그 · 내부 실행 · 검사

docker logs — 컨테이너 로그

명령어	설명
<code>docker logs mablog_proxy</code>	전체 로그
<code>docker logs mablog_proxy --tail 20</code>	마지막 20줄만
<code>docker logs mablog_proxy -f</code>	실시간 로그 follow
<code>docker logs mablog_proxy 2>&1</code>	stdout + stderr 모두 포함
<code>docker logs mablog_proxy --tail 60 2>&1 grep ERROR</code>	에러 줄만 필터

```
# 로그에서 ERROR 찾기
docker logs mablog_logstash 2>&1 | grep -E "ERROR|WARN" | head -20
```

docker exec — 컨테이너 내부에서 명령 실행

명령어	설명
<code>docker exec 컨테이너명 명령어</code>	컨테이너 안에서 명령 실행
<code>docker exec -it 컨테이너명 bash</code>	대화형 bash 셸 접속 (-i=interactive, -t=tty)
<code>docker exec -it 컨테이너명 sh</code>	bash 없을 때 sh로

```
# MongoDB 백업 (컨테이너 내부에서 실행)
docker exec mablog_mongodb mongodump --username mongoAdmin --password 비번 \
  --authenticationDatabase admin --out /tmp/mablog_dump

# Elasticsearch 상태 확인
docker exec mablog_elasticsearch curl -s http://localhost:9200/_cluster/health

# Logstash 플러그인 목록
docker exec mablog_logstash bin/logstash-plugin list | grep kafka
```

docker cp — 컨테이너와 파일 복사

명령어	설명
<code>docker cp 컨테이너명:/컨테이너경로 /호스트경로</code>	컨테이너 → 호스트
<code>docker cp /호스트경로 컨테이너명:/컨테이너경로</code>	호스트 → 컨테이너

```
# mongodump 결과물을 컨테이너 밖으로 꺼내기
docker cp mablog_mongodb:/tmp/mablog_dump /tmp/mablog_dump_host
```

docker exec로 컨테이너 안에서 만든 파일은 컨테이너 파일시스템에만 존재. 호스트에서 tar로 압축하려면 먼저 docker cp로 꺼내야 함.

docker inspect — 컨테이너 상세 정보

```
# 마운트된 볼륨 확인
docker inspect mablog_proxy --format '{{json .Mounts}}'

# 컨테이너 IP 확인
docker inspect mablog_nextjs --format '{{.NetworkSettings.Networks.mablog_network.IPAddress}}'
```

7. Docker Compose

기본 명령어

명령어	설명
<code>docker-compose up -d</code>	전체 서비스 시작 (d=백그라운드)
<code>docker-compose up -d 서비스명</code>	특정 서비스만 시작/재생성
<code>docker-compose down</code>	전체 서비스 중지 + 컨테이너 삭제
<code>docker-compose restart 서비스명</code>	재시작 (설정 변경 반영 안 됨)
<code>docker-compose ps</code>	서비스 상태 목록
<code>docker-compose logs -f 서비스명</code>	실시간 로그
<code>docker-compose build</code>	이미지 빌드 (Dockerfile 변경 시)

restart vs up -d 차이

`docker-compose restart` : 컨테이너를 꺾다 컴. docker-compose.yml 변경 미반영.

`docker-compose up -d` : yml과 비교해서 달라진 게 있으면 컨테이너를 새로 만들. 볼륨/환경 변수 변경은 반드시 이걸 써야 반영됨.

docker-compose.yml 핵심 구조

```
services:
  서비스명:
    image: nginx:stable-alpine           # 사용할 이미지
    container_name: mablog_proxy         # 컨테이너 이름
    restart: always                      # 죽으면 자동 재시작
    ports:
      - "80:80"                         # 호스트:컨테이너 포트 매핑
      - "127.0.0.1:27017:27017"         # 로컬호스트만 노출
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro  # 호스트경로:컨테이너경로:옵션
    environment:
      - NODE_ENV=production
    depends_on:
      - db                                # db 서비스 먼저 시작 후 실행
    networks:
      mablog_network:
```

```
aliases:
  - elasticsearch          # 네트워크 내 별명 추가

networks:
  mablog_network:
    driver: bridge
```

8. Git

기본 워크플로우

명령어	설명
git status	변경된 파일 목록
git diff	변경 내용 상세 보기
git add 파일명	스테이징 (커밋 준비)
git add -A	전체 변경 파일 스테이징
git commit -m "메시지"	커밋 생성
git push	원격 저장소에 올리기
git pull	원격 저장소에서 가져오기
git log --oneline	커밋 히스토리 한 줄씩

원격 저장소 관리

```
# 원격 URL 확인
git remote -v

# 원격 URL 변경 (PAT 토큰 만료 시)
git remote set-url origin https://PasserbyMa:ghp_토큰값@github.com/PasserbyMa/madayblog.git

# 브랜치 확인
git branch -a
```

GitHub Actions 핵심 개념

```
# .github/workflows/auto-post.yml
on:
  push:
    branches: [main]      # main 브랜치에 push될 때 실행

jobs:
  write-post:
    runs-on: ubuntu-latest # GitHub 서버에서 Ubuntu로 실행
    steps:
```



```
- uses: actions/checkout@v4      # 코드 체크아웃
- run: node script.mjs           # 스크립트 실행
env:
  API_KEY: ${ secrets.API_KEY }  # GitHub Secret 사용
```

9. 기타 도구

apt — 패키지 설치

명령어	설명
<code>sudo apt-get update</code>	패키지 목록 업데이트
<code>sudo apt-get install -y 패키지명</code>	패키지 설치 (-y: 자동 yes)
<code>sudo apt-get remove 패키지명</code>	패키지 제거

```
sudo apt-get install -y cron # cron 설치
```

npm — Node.js 패키지 관리

명령어	설명
<code>npm install 패키지명</code>	패키지 설치 (package.json에 추가)
<code>npm install</code>	package.json의 모든 의존성 설치
<code>npm run build</code>	빌드 스크립트 실행
<code>npm run dev</code>	개발 서버 시작

weasyprint — HTML → PDF 변환

```
weasyprint input.html output.pdf
```

which — 명령어 위치 확인

```
which pandoc # → /usr/bin/pandoc
which node # → /usr/bin/node
```

파이프(|)와 리다이렉션(>, >>)

기호	의미	예시
	앞 명령 출력을 뒤 명령 입력으로	<code>ps aux grep nginx</code>

>	출력을 파일로 (덮어쓰기)	echo "hi" > file.txt
>>	출력을 파일로 (이어쓰기)	echo "hi" >> log.txt
2>&1	stderr를 stdout으로 합침	cmd 2>&1 grep error
/dev/null	출력 버리기	cmd -o /dev/null

10. 자주 쓰는 조합 패턴

서버 상태 한 번에 확인

```
free -h && echo "---" && df -h && echo "---" && docker stats --no-stream
```

컨테이너 전체 상태

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

특정 컨테이너 로그에서 에러 찾기

```
docker logs mablog_logstash 2>&1 | grep -E "ERROR|FATAL" | tail -20
```

Elasticsearch 인덱스 확인

```
docker exec mablog_elasticsearch curl -s "http://localhost:9200/_cat/indices?v"
```

nginx 로그 실시간 보기

```
docker logs mablog_proxy -f 2>&1  
# 또는 파일로 저장된 경우  
tail -f /home/ubuntu/mam/nginx_logs/access.log
```

디스크 많이 쓰는 디렉토리 찾기

```
du -sh /home/ubuntu/mam/* | sort -rh | head -10  
# du: 디렉토리 크기, sort -rh: 큰 것부터, head -10: 상위 10개
```

포트 충돌 확인

```
ss -tulnp | grep :80      # 80번 포트 사용 중인 프로세스
```